

Formal Definitions: Adversarial Training and Saliency Evaluation for DNA Sequence Models

1 Notation

Let $\mathbf{x} \in \{0, 1\}^{4 \times L}$ denote a one-hot encoded DNA sequence of length L , where each position $\mathbf{x}_{:,i} \in \{0, 1\}^4$ represents one of the four nucleotides $\{A, C, G, T\}$. Let $f_\theta : \mathbb{R}^{4 \times L} \rightarrow \mathbb{R}$ be a neural network mapping sequences to logits, and $\mathcal{L}(f_\theta(\mathbf{x}), y)$ the binary cross-entropy loss for label $y \in \{0, 1\}$.

For a sequence with a planted motif, let $\mathcal{M} \subset \{1, \dots, L\}$ denote the set of positions within the ground-truth motif region.

2 Adversarial Training Methods

2.1 Direct HotFlip: Gradient-Based Adversarial Training

Direct HotFlip generates adversarial perturbations by flipping nucleotides that maximally increase the loss, as determined by input gradients.

Definition 1 (Flip Saliency). For a sequence $\mathbf{x}$ at position i currently encoding nucleotide $b = \arg \max_j \mathbf{x}_{j,i}$, the *flip saliency* for substituting to nucleotide $b' \neq b$ is:

$$S_{b',i}(\mathbf{x}) = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{b',i}} - \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{b,i}} \quad (1)$$

This measures the first-order effect on loss of flipping position i from b to b' .

Algorithm 1 Direct HotFlip Adversarial Example Generation

Require: Sequence $\mathbf{x}$, label y , model f_θ , number of flips k

Ensure: Adversarial sequence $\tilde{\mathbf{x}}$

- 1: Compute gradient $\mathbf{G} \leftarrow \nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y)$
 - 2: Compute all flip saliencies $S_{b',i}$ for valid (b', i) pairs
 - 3: $\mathcal{F} \leftarrow \text{top-}k\{(b', i) : S_{b',i}\}$ $\{k$ highest saliency flips $\}$
 - 4: $\tilde{\mathbf{x}} \leftarrow \mathbf{x}$
 - 5: **for** each $(b^*, i^*) \in \mathcal{F}$ **do**
 - 6: $\tilde{\mathbf{x}}_{:,i^*} \leftarrow \mathbf{e}_{b^*}$
 - 7: **end for**
 - 8: **return** $\tilde{\mathbf{x}}$
-

The number of flips k is determined by the *flip fraction* $\epsilon \in (0, 1)$ as $k = \lfloor \epsilon \cdot L \rfloor$.

Flip Scheduling. During training, the effective flip fraction can be scheduled to gradually increase adversarial strength:

$$\epsilon_t = \epsilon_{\max} \cdot \frac{t}{T} \quad (2)$$

where t is the current epoch and T is the total number of epochs.

2.2 Iterative HotFlip (Variant)

Iterative HotFlip refines the perturbation by recomputing gradients after each flip, potentially finding more effective adversarial examples at higher computational cost.

Algorithm 2 Iterative HotFlip Adversarial Example Generation

Require: Sequence $\mathbf{x}$, label y , model f_θ , number of flips k

Ensure: Adversarial sequence $\tilde{\mathbf{x}}$

```

1:  $\tilde{\mathbf{x}} \leftarrow \mathbf{x}$ 
2: for  $t = 1, \dots, k$  do
3:   Compute gradient  $\mathbf{G} \leftarrow \nabla_{\tilde{\mathbf{x}}} \mathcal{L}(f_\theta(\tilde{\mathbf{x}}), y)$ 
4:   for each position  $i$  and each nucleotide  $b' \neq \arg \max_j \tilde{\mathbf{x}}_{j,i}$  do
5:      $S_{b',i} \leftarrow G_{b',i} - G_{b,i}$  where  $b = \arg \max_j \tilde{\mathbf{x}}_{j,i}$ 
6:   end for
7:    $(b^*, i^*) \leftarrow \arg \max_{b',i} S_{b',i}$  {Best flip}
8:    $\tilde{\mathbf{x}}_{:,i^*} \leftarrow \mathbf{e}_{b^*}$  {Apply flip}
9: end for
10: return  $\tilde{\mathbf{x}}$ 

```

Iterative HotFlip requires k forward-backward passes compared to one for Direct HotFlip, but accounts for gradient changes induced by earlier flips.

2.3 Random Smoothing (Dirichlet)

Random smoothing replaces discrete one-hot inputs with continuous relaxations sampled from position-wise Dirichlet distributions centered on the original nucleotides.

Definition 2 (Dirichlet Smoothing). For a one-hot vector $\mathbf{x}_{:,i} = \mathbf{e}_b$ at position i , the smoothed input is sampled as:

$$\tilde{\mathbf{x}}_{:,i} \sim \text{Dir}(\boldsymbol{\alpha}^{(i)}), \quad \text{where } \alpha_j^{(i)} = \begin{cases} \alpha_+ & \text{if } j = b \\ 1 & \text{otherwise} \end{cases} \quad (3)$$

The concentration parameter α_+ controls the expected perturbation magnitude.

Parameterization by Expected Perturbation. Given a target expected perturbation $\epsilon \in (0, 1)$, defined as the expected probability mass on non-original nucleotides:

$$\epsilon = 1 - \mathbb{E}[\tilde{\mathbf{x}}_{b,i}] = 1 - \frac{\alpha_+}{\alpha_+ + 3} \quad (4)$$

Solving for α_+ :

$$\alpha_+ = \frac{3(1 - \epsilon)}{\epsilon} \quad (5)$$

Training proceeds by sampling $\tilde{\mathbf{x}}$ for each input $\mathbf{x}$ and optimizing $\mathcal{L}(f_\theta(\tilde{\mathbf{x}}), y)$.

2.4 Gaussian Smoothing

Gaussian smoothing adds isotropic Gaussian noise directly to the one-hot encoded inputs without re-normalization.

Definition 3 (Gaussian Input Smoothing). For input $\mathbf{x} \in \mathbb{R}^{4 \times L}$, the smoothed input is:

$$\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}) \quad (6)$$

where σ^2 is the noise variance.

Unlike Dirichlet smoothing, Gaussian smoothing does not preserve the simplex constraint $\sum_j \tilde{\mathbf{x}}_{j,i} = 1$.

3 Boundary Integrated Gradients

Integrated Gradients (IG) attributes model predictions to input features by accumulating gradients along a path from a baseline to the input. We introduce *Boundary IG*, which uses an adversarially-generated baseline and applies gradient correction for DNA sequences.

3.1 PGD Baseline Generation

For a correctly classified input $(\mathbf{x}, y)$, we seek a nearby point $\mathbf{x}'$ where the model prediction flips. This provides a semantically meaningful baseline representing the “decision boundary.”

Algorithm 3 PGD Baseline Search

Require: Input $\mathbf{x}$, label y , model f_θ , iterations T , step size η , perturbation bound δ

Ensure: Baseline $\mathbf{x}'$ (or $\mathbf{x}^{(u)}$ if unsuccessful)

```

1:  $\mathbf{x}' \leftarrow \mathbf{x}$ 
2: for  $t = 1, \dots, T$  do
3:    $\mathbf{g} \leftarrow \nabla_{\mathbf{x}'} \mathcal{L}(f_\theta(\mathbf{x}'), y)$ 
4:    $\mathbf{x}' \leftarrow \mathbf{x}' + \eta \cdot \text{sign}(\mathbf{g})$  {Maximize loss}
5:    $\mathbf{x}' \leftarrow \text{clip}(\mathbf{x}', \mathbf{x} - \delta, \mathbf{x} + \delta)$  {Project to  $\ell_\infty$  ball}
6:    $\mathbf{x}' \leftarrow \text{clip}(\mathbf{x}', 0, 1)$  {Ensure valid range}
7:   if  $\text{sign}(f_\theta(\mathbf{x}')) \neq \text{sign}(f_\theta(\mathbf{x}))$  then
8:     return  $\mathbf{x}'$  {Prediction flipped}
9:   end if
10: end for
11: return  $\mathbf{x}^{(u)}$  {Fallback: uniform baseline}
```

The fallback baseline $\mathbf{x}^{(u)}$ is the uniform distribution over nucleotides:

$$\mathbf{x}_{j,i}^{(u)} = 0.25 \quad \forall j \in \{1, 2, 3, 4\}, i \in \{1, \dots, L\} \quad (7)$$

This represents a position with no nucleotide information, providing a semantically meaningful reference point for DNA sequences.

3.2 Gradient-Corrected Integrated Gradients

Standard IG computes attributions by integrating gradients along a straight-line path from baseline $\mathbf{x}'$ to input $\mathbf{x}$:

$$\text{IG}_j(\mathbf{x}) = (\mathbf{x}_j - \mathbf{x}'_j) \int_0^1 \frac{\partial f_\theta(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial \mathbf{x}_j} d\alpha \quad (8)$$

For DNA sequences, we apply a *gradient correction* that removes the mean gradient across nucleotide channels at each position, ensuring attributions reflect relative importance within each position.

Definition 4 (Gradient-Corrected IG for DNA). Let $\mathbf{A} \in \mathbb{R}^{4 \times L}$ be the raw IG attribution tensor. The corrected attributions are:

$$\tilde{A}_{j,i} = A_{j,i} - \frac{1}{4} \sum_{k=1}^4 A_{k,i} \quad (9)$$

The position-wise attribution score is then:

$$a_i = \left| \sum_{j=1}^4 \tilde{A}_{j,i} \cdot \mathbf{x}_{j,i} \right| \quad (10)$$

This computes the contribution of the actual nucleotide at each position under the corrected gradients.

Definition 5 (Boundary Integrated Gradients). Boundary IG combines PGD baseline generation with gradient-corrected IG:

1. Generate baseline $\mathbf{x}'$ via Algorithm 3
2. Compute raw IG: $\mathbf{A} = \text{IG}(\mathbf{x}; \mathbf{x}')$
3. Apply gradient correction to obtain $\tilde{\mathbf{A}}$
4. Compute position-wise scores $\mathbf{a} = (a_1, \dots, a_L)$

4 Evaluation Metrics

Let $\mathbf{a} = (a_1, \dots, a_L)$ be the position-wise attribution scores and $\mathcal{M} \subset \{1, \dots, L\}$ the ground-truth motif region.

4.1 Saliency AUC

Saliency AUC measures how well attribution scores discriminate motif positions from background positions, analogous to the area under the ROC curve.

Definition 6 (Saliency AUC).

$$\text{SaAUC} = \frac{1}{|\mathcal{M}| \cdot |\bar{\mathcal{M}}|} \sum_{i \in \mathcal{M}} \sum_{j \in \bar{\mathcal{M}}} \mathbf{1}[a_i > a_j] \quad (11)$$

where $\bar{\mathcal{M}} = \{1, \dots, L\} \setminus \mathcal{M}$ is the complement (background positions).

Saliency AUC equals 1.0 when all motif positions have higher attribution than all background positions, and 0.5 for random attributions.

4.2 Windowed Intersection-over-Union (wIoU)

wIoU measures localization accuracy by finding the contiguous window of fixed length k (matching the motif length) that maximizes total attribution, then computing its overlap with the ground truth.

Definition 7 (Windowed IoU). Let $k = |\mathcal{M}|$ be the motif length. Define the predicted region as:

$$\hat{\mathcal{M}} = \{i^*, i^* + 1, \dots, i^* + k - 1\}, \quad \text{where } i^* = \arg \max_i \sum_{j=i}^{i+k-1} a_j \quad (12)$$

The windowed IoU is:

$$\text{wIoU} = \frac{|\hat{\mathcal{M}} \cap \mathcal{M}|}{|\hat{\mathcal{M}} \cup \mathcal{M}|} \quad (13)$$

4.3 Saliency Signal-to-Noise Ratio (SaSNR)

SaSNR quantifies the concentration of attribution energy within the motif region.

Definition 8 (Saliency SNR).

$$\text{SaSNR} = \frac{\sum_{i \in \mathcal{M}} a_i^2}{\sum_{i=1}^L a_i^2} \quad (14)$$

SaSNR equals 1.0 when all attribution is concentrated in the motif and equals $|\mathcal{M}|/L$ for uniform attributions.

4.4 Expected SaSNR Under Null

For statistical comparison, we compute the expected SaSNR under a random attribution baseline.

Definition 9 (Expected SaSNR). If attributions were uniformly distributed across positions:

$$\mathbb{E}[\text{SaSNR}] = \frac{|\mathcal{M}|}{L} \quad (15)$$

This serves as a null baseline; effective attribution methods should achieve $\text{SaSNR} \gg \mathbb{E}[\text{SaSNR}]$.

5 Training Objective

5.1 Phase 1: Standard Training

In Phase 1, the model is trained on clean inputs to maximize classification accuracy:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\mathcal{L}(f_{\theta}(\mathbf{x}), y)] \quad (16)$$

5.2 Phase 2: Robust Training

In Phase 2, the architecture is fixed from Phase 1 and the model is trained on adversarially perturbed inputs. For HotFlip-based methods:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\mathcal{L}(f_{\theta}(\tilde{\mathbf{x}}), y)] \quad (17)$$

where $\tilde{\mathbf{x}}$ is generated by Algorithm 2 or 1.

The optimization objective combines accuracy and interpretability:

$$\text{Objective} = 0.9 \cdot \text{SaAUC} + 0.1 \cdot \text{Accuracy} \quad (18)$$